Below is the **standard Church-encoding** of lists in System F, written to match *exactly the style and conventions of your development*.
I also walk through the reasoning so you see how the type arises.

🎯 Goal

Define

Definition List (A : Ty) : Ty

together with constructors

nil : Tm (List A)

cons : Tm (A ⇒ List A ⇒ List A)

and the eliminator

iteList : ∀{X}, Tm X → Tm (A ⇒ List A ⇒ X) → Tm (List A ⇒ X)

and show why the type definition has the form it does.

---

1. 🧠 Principle (Church encoding)

All your inductive encodings follow this pattern:

An inductive type is the polymorphic type of its folds

$$\mu F \triangleq \forall X.\,(F\,X \to X) \to X$$

Concretely,

- Bool = ∀ A, A ⇒ A ⇒ A
- Nat = ∀ A, A ⇒ (A ⇒ A) ⇒ A
- Prod A B = ∀ X, (A ⇒ B ⇒ X) ⇒ X

For a **list** of elements of type A, the algebra for folding consists of:

- a value for the empty list
  nilCase : X
- a function for the cons case
  consCase : A ⇒ X ⇒ X

Thus a list is exactly a function that, given such an algebra, produces an X.

So the Church type of lists is:

$$\mathrm{List}(A) = \forall X, X \Rightarrow (A \Rightarrow X \Rightarrow X) \Rightarrow X.$$

This matches Nat except that the step function also receives the head.

---

2. ✍️ Definition of the type

Definition List (A : Ty) : Ty := ∀λ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X.

---

3. ✔️ Constructors

nil

For the empty list, given algebra (z : X) and (c : A ⇒ X ⇒ X), we return z.

Definition nil {A : Ty} : Tm (List A) := Λ X, λ z, λ c, z.

cons

For cons a ℓ, given an algebra, we compute:

cons a ℓ • X · z · c $\stackrel{\text{def}}{=}$ c · a · (ℓ • X · z · c)

Definition cons {A : Ty} : Tm (A ⇒ List A ⇒ List A) := λ a, λ l, Λ X, λ z, λ c, c · a · (l • X · z · c).

---

4. 🧩 Eliminator / iterator (fold)

Exactly like iteNat, but with the list shape:

Definition iteList {A X : Ty} (z : Tm X) (c : Tm (A ⇒ X ⇒ X)) (l : Tm (List A)) : Tm X := l • X · z · c.

If you want it curried like iteNat, you can write:

Definition foldList {A X : Ty} (z : Tm X) (c : Tm (Tm A -> Tm X -> Tm X)) (l : Tm (List A)) : Tm X := l • X · z · lam (fun a => lam (fun r => c a r)).

---

5. β-laws

For nil

iteList z c nil

= (Λ X, λ z, λ c, z) • X · z · c

= z.

Exactly what you expect.

For cons

iteList z c (cons a l)

= (Λ X, λ z, λ c, c · a · (l • X · z · c)) • X · z · c

= c · a · (iteList z c l).

---

6. 📦 Final consolidated code

(* List A = ∀ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X *) Definition List (A : Ty) : Ty := ∀λ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X. (* nil constructor *) Definition nil {A : Ty} : Tm (List A) := Λ X, λ z, λ c, z. (* cons constructor *) Definition cons {A : Ty} : Tm (A ⇒ List A ⇒ List A) := λ a, λ l, Λ X, λ z, λ c, c · a · (l • X · z · c). (* eliminator / fold *) Definition iteList {A X : Ty} (z : Tm X) (c : Tm (A ⇒ X ⇒ X)) (l : Tm (List A)) : Tm X := l • X · z · c.

If you want, I can also provide:

- proofs of the β-laws for nil and cons
- definitions of map, append, and length
- alternative encodings (Scott, Mendler, Bohm–Berarducci)

Just tell me!

Ezt mondtad:

Thank you! The cons is defined this way: Definition cons {A} (x : Tm A) (xs : Tm (List A)) : Tm (List A) Could you modify the definition and the following definitions to reflect that please?

A ChatGPT ezt mondta:

Absolutely — here is the **exact same Church encoding**, but rewritten so that

cons {A} : Tm A → Tm (List A) → Tm (List A)

rather than being a curried lambda living in a function type.

Everything else (nil, List, iteList) is updated to match.

✓ Final version with the requested cons signature

1. Type of lists

Definition List (A : Ty) : Ty := ∀λ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X.

---

2. Constructors

nil

Definition nil {A : Ty} : Tm (List A) := Λ X, λ z, λ c, z.

---

cons **(as requested: takes its arguments explicitly)**

Definition cons {A : Ty} (x : Tm A) (xs : Tm (List A)) : Tm (List A) := Λ X, λ z, λ c, (* apply cons-algebra element: c : A ⇒ X ⇒ X *) c · x · (xs • X · z · c).

This matches your desired interface exactly:

cons : Tm A → Tm (List A) → Tm (List A)

---

3. Eliminator / iterator

Same logic as before, but rewritten to match the above form:

Definition iteList {A X : Ty} (z : Tm X) (c : Tm (A ⇒ X ⇒ X)) (l : Tm (List A)) : Tm X := l • X · z · c.

(iteList is still just instantiating the Church-encoded list.)

---

4. β-laws (still hold)

nil**:**

iteList z c (nil)

= z.

cons**:**

iteList z c (cons x xs)

= c · x · (iteList z c xs).

Both follow immediately from rewriting with Forallβ and arrβ.

---

5. Full block (copy-paste ready)

(* Church-encoded lists *) Definition List (A : Ty) : Ty := ∀λ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X.
Definition nil {A : Ty} : Tm (List A) := Λ X, λ z, λ c, z. Definition cons {A : Ty} (x : Tm A) (xs : Tm

(List A)) : Tm (List A) := Λ X, λ z, λ c, c · x · (xs • X · z · c). Definition iteList {A X : Ty} (z : Tm X) (c : Tm (A ⇒ X ⇒ X)) (l : Tm (List A)) : Tm X := l • X · z · c.

If you want, I can also give you:

- map : (A ⇒ B) → List A → List B
- append : List A → List A → List A
- length : List A → Nat

Just say the word!

Ezt mondtad:

Thank you again! And what if the iterator is defined this way? Definition iteList {A B : Ty} (z : Tm B) (s : Tm A -> Tm B -> Tm B) (xs : Tm (List A)) : Tm B

A ChatGPT ezt mondta:

Great — if your iterator has the *non-polymorphic*, higher-order Coq type

Definition iteList {A B : Ty} (z : Tm B) (s : Tm A -> Tm B -> Tm B) (xs : Tm (List A)) : Tm B.

then we just need to translate the step function

s : Tm A → Tm B → Tm B

into a *System-F term*

c : Tm (A ⇒ B ⇒ B)

which is exactly the Church-encoded fold algebra expects.

We do this by **lambda-lifting**:

lam (fun a => lam (fun r => s a r))

which has type Tm (A ⇒ B ⇒ B).

Then we simply feed z and the lifted step function to the Church-encoded list:

✓ **Final definition of** iteList

Definition iteList {A B : Ty} (z : Tm B) (s : Tm A -> Tm B -> Tm B) (xs : Tm (List A)) : Tm B := xs • B · z · (lam (fun a => lam (fun r => s a r))).

🧠 **Why this is correct**

Given the Church encoding:

List A = ∀λ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X

a list xs : Tm (List A) expects:

1. z : Tm X (nil case)
2. c : Tm (A ⇒ X ⇒ X) (cons case)

So for your signature:

iteList z s xs

we use:

- X := B
- z as given
- c := lam (fun a => lam (fun r => s a r))

β-laws (still work!)

For nil**:**

iteList z s nil

= z.

For cons x xs**:**

iteList z s (cons x xs)

= s x (iteList z s xs).

Both follow directly from β-reduction and the definition of cons.

📦 **Full block (copy-paste ready)**

Definition List (A : Ty) : Ty := ∀λ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X. Definition nil {A : Ty} : Tm (List A) := Λ X, λ z, λ c, z. Definition cons {A : Ty} (x : Tm A) (xs : Tm (List A)) : Tm (List A) := Λ X, λ z, λ c, c · x · (xs • X · z · c). Definition iteList {A B : Ty} (z : Tm B) (s : Tm A -> Tm B -> Tm B) (xs : Tm (List A)) : Tm B := xs • B · z · (lam (fun a => lam (fun r => s a r))).

If you'd like, I can now also define:

- map using this iterator

- append
- foldr
- head, tail
- induction principles

Just let me know!

Ezt mondtad:

Isn't there an other way to define the iterator? In a similar way to how it was done for Bool and Nat.

A ChatGPT ezt mondta:

Yes — **there *is* a way to define the iterator exactly like the Bool and Nat cases**, i.e. without passing a *meta-level* function
(Tm A → Tm B → Tm B)
but instead passing a *System-F term*

s : Tm (A ⇒ B ⇒ B).

This gives you an iterator fully *inside System F*, just like:

- iteBool t f b = b • A · t · f
- iteNat z s n = n • A · z · lam s (where s : Tm A → Tm A)

For lists, the iterator becomes:

iteList z s xs = xs • B · z · s

with **no lambda-lifting**.

✓ Iterator version fully inside System F

If we define:

- z : Tm B
- s : Tm (A ⇒ B ⇒ B)
- xs : Tm (List A)

then the iterator is simply:

Definition iteList {A B : Ty} (z : Tm B) (s : Tm (A ⇒ B ⇒ B)) (xs : Tm (List A)) : Tm B := xs • B · z · s.

This exactly parallels iteBool and iteNat.

✓ Why this matches Bool and Nat

Compare:

Bool

iteBool t f b := b • A · t · f.

Nat

iteNat z s n := n • A · z · lam s.

- z : Tm A
- but s : Tm A → Tm A, so we had to wrap it using
  lam s : Tm (A ⇒ A).

List

List expects:

xs : ∀ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X

Thus:

xs • B · z · s : Tm B

with:

- z : Tm B
- s : Tm (A ⇒ B ⇒ B) ← already a System-F term, so **no lambda lifting required**

This matches the "shape" of Bool and Nat perfectly.

✓ Updated constructors (unchanged)

Definition List (A : Ty) : Ty := ∀λ X, X ⇒ (A ⇒ X ⇒ X) ⇒ X. Definition nil {A : Ty} : Tm (List A) := Λ X, λ z, λ c, z. Definition cons {A : Ty} (x : Tm A) (xs : Tm (List A)) : Tm (List A) := Λ X, λ z, λ c, c · x · (xs • X · z · c).

✓ Iterator (System-F version)

Definition iteList {A B : Ty} (z : Tm B) (s : Tm (A ⇒ B ⇒ B)) (xs : Tm (List A)) : Tm B := xs • B · z · s.

✓ β-laws (cleanest possible form)

Nil

iteList z s nil = z.

Cons

iteList z s (cons x xs) = s · x · (iteList z s xs).

These follow immediately by reducing the Church encoding.

✓ Summary

There are two iterator styles:

Style	**Step function type**	**Matches Bool/Nat?**	**Definition**
System-F iterator	s : Tm (A ⇒ B ⇒ B)	✅ **YES**	xs • B · z · s
Meta-level iterator	s : Tm A → Tm B → Tm B	❌ Needs lambda lifting	xs • B · z · lam …

So yes — the first one is the *canonical* Church-encoding iterator and the most elegant.

If you want, I can also give you:

- map, append, length using this iterator
- the induction principle
- proofs of the β-laws in Coq

Just let me know!

Csatolás

Keresés

Tanulás

Kép létrehozása

Hang

A ChatGPT hibázhat. Fontold meg a fontos információk ellenőrzését. Lásd Cookie beállítások.